

HLG LLM Utils: An Advanced Graph RAG Framework with Multi-Agent Orchestration, Hierarchical Leiden Community Detection, and Cypher Query Self-Correction

Nguyen Hoai Nam

VNUHCM – University of Science
Faculty of Data Science
Student ID: 25C01014

Pham Cung Le Nhan Vu

VNUHCM – University of Science
Faculty of Data Science
Student ID: 25C01049

Abstract

Standard Retrieval-Augmented Generation (RAG) systems based on flat chunking and vector similarity suffer from global context fragmentation and multi-hop reasoning failures. We present **HLG LLM Utils**, a modular Graph RAG framework built on a Python uv-workspace monorepo comprising 10 packages. Our system constructs a Neo4j Property Knowledge Graph through LLM-driven entity, relationship, and claims extraction with iterative Gleaning refinement. We implement four complementary retrieval strategies: (1) **Local Search** combining vector similarity with 1-hop graph traversal and cross-encoder reranking; (2) **Global Search** using Hierarchical Leiden community detection with parallel Map-Reduce LLM summarization; (3) a **Cypher Self-Correction State Machine** that translates natural language to executable Cypher with a 3-iteration error feedback loop; and (4) **ARK** (Adaptive Retriever of Knowledge), a multi-agent parallel stochastic graph explorer with Voting Rank Fusion (VRF). The system is deployed as a full-stack application with a FastAPI backend, React 19 frontend, self-hosted nomic-embed-text-v1.5 embeddings via vLLM, and PaddleOCR-VL for document parsing, orchestrated via Docker Compose.

1 Introduction

Large Language Models (LLMs) have achieved remarkable proficiency in natural language understanding and generation. However, standalone LLMs remain susceptible to hallucinations and knowledge cutoff constraints because their parametric knowledge is static (Lewis et al., 2020). Retrieval-Augmented Generation (RAG) addresses this by grounding LLM responses in external

data sources, but Naive RAG—relying on sliding-window chunking and cosine similarity—exhibits fundamental limitations.

We identify three critical failure modes:

1. **Structural Context Fragmentation:** Fixed-size chunking severs conceptual relationships spanning across paragraphs or documents.
2. **Multi-Hop Reasoning Blindspots:** Vector similarity retrieves chunks mentioning query terms but misses crucial intermediate relational links in chains $A \rightarrow B \rightarrow C$.
3. **Corpus-Wide Summarization Failures:** Embedding similarity cannot answer high-level thematic queries that require synthesis across the entire corpus.

To address these limitations, we design and implement **HLG LLM Utils**, a production-grade Graph RAG framework combining Property Knowledge Graphs with multi-agent coordination and hierarchical community detection. Our contributions include:

- A **Property Knowledge Graph** schema persisted in Neo4j with automated entity, relationship, and claims extraction via LLM structured output, including iterative Gleaning refinement.
- **Hierarchical Leiden Community Detection** with multi-level resolution scaling and LLM-generated community summaries enabling Map-Reduce Global Search.
- A **LangGraph Cypher Self-Correction State Machine** with automatic 3-iteration error recovery against Neo4j execution logs.

- Integration of the **ARK** algorithm (Polonuer et al., 2026) for autonomous multi-agent graph exploration with Voting Rank Fusion.
- A **modular monorepo architecture** (10 uv workspace packages) with full-stack deployment via Docker Compose.

2 Related Work

Knowledge Graphs and Representation. Knowledge Representation has evolved from semantic networks (Quillian, 1968) and frame systems (Minsky, 1974) through Description Logics (Baader et al., 2003) to modern Property Knowledge Graphs (Robinson et al., 2015). Ji et al. (2021) survey KG construction and application, while Pan et al. (2024) provide a roadmap for unifying LLMs with Knowledge Graphs, establishing the foundation for our LLM-driven extraction pipeline.

Retrieval-Augmented Generation. Lewis et al. (2020) introduced RAG for knowledge-intensive NLP tasks, combining dense retrieval with sequence-to-sequence generation. Subsequent work has explored hybrid retrieval combining sparse BM25 (Robertson et al., 2009) with dense vectors through Reciprocal Rank Fusion (Cormack et al., 2009). Our system implements this hybrid approach with configurable sparse-dense weighting.

Graph RAG and Community Detection. Edge et al. (2024) demonstrated that hierarchical community summarization enables effective global queries over large corpora. The Leiden algorithm (Traag et al., 2019) guarantees well-connected communities through local sub-community refinement, addressing known issues with the Louvain method. Our system implements multi-level Leiden detection with dual backends (Neo4j GDS and Python igraph+leidenalg).

Adaptive Knowledge Graph Retrieval. Polonuer et al. (2026) proposed ARK, an agentic framework providing LLMs with two complementary tools—global lexical search for breadth-oriented discovery and neighborhood exploration for depth-oriented multi-hop traversal—enabling the model to autonomously navigate knowledge graphs. Our implementation extends ARK with parallel stochastic trajectories and Voting Rank Fusion.

Self-Refinement and Error Correction. Madaan et al. (2023) established the paradigm of iterative self-refinement with self-feedback for LLM outputs. We apply this principle to both knowledge extraction (Gleaning) and Cypher query generation (self-correction state machine).

3 Dataset

HLG LLM Utils is designed as a domain-agnostic framework that constructs its knowledge graph dynamically from user-uploaded documents. The system accepts multiple input formats:

- **Document Types:** PDF, TXT, Markdown, DOCX (max 50MB per file).
- **Parsing Pipeline:** Multi-format text extraction via PyMuPDF with OCR fallback chain: PaddleOCR-VL-1.5 (primary, served via vLLM on port 8083) → Tesseract → Docling.
- **Storage:** Raw files are stored in MinIO S3-compatible object storage; metadata (filename, status, entity/relationship counts) is tracked in PostgreSQL.

Upon ingestion, documents are chunked using configurable strategies (standard recursive splitting at 1,000 tokens with 200-token overlap by default) and processed through the entity extraction pipeline. The resulting knowledge graph is persisted in Neo4j 5.18 with APOC plugin support. Embeddings are generated using self-hosted nomic-ai/nomic-embed-text-v1.5 served via vLLM.

4 Method

4.1 Property Knowledge Graph Construction

We model the Property Knowledge Graph as $\mathcal{G} = (V, E, \mathcal{L}_V, \mathcal{R}_E, \Phi_V, \Phi_E)$, where V is the set of entity vertices, $E \subseteq V \times \mathcal{R}_E \times V$ is the set of directed typed edges, $\mathcal{L}_V = \{\text{Document, Chunk, Entity, Community, Claim}\}$ are node labels, $\mathcal{R}_E = \{\text{HAS_CHUNK, MENTIONS, RELATED_TO, BELONGS_TO, PARENT_OF, HAS_CHILD}\}$ are relationship types, and Φ_V, Φ_E map property attributes onto vertices and edges respectively.

4.1.1 Entity and Relationship Extraction

For each text chunk C_k , the system invokes the LLM with structured output via Pydantic schemas:

$$T_{\text{extract}}(C_k) \rightarrow \left(\{e_i\}_{i=1}^{N_e}, \{r_{ij}\}_{j=1}^{N_r} \right) \quad (1)$$

where each entity $e_i = (\text{id}_i, \text{type}_i, \text{desc}_i)$ and each relationship $r_{ij} = (e_i, \text{rel_type}, e_j, \text{desc}_{ij})$. Entity IDs are normalized via $\text{id}_{\text{upper}} = \text{UPPER}(\text{TRIM}(\text{id}_i))$, and relationship types are sanitized using regex: $\text{re.sub}(r"[^A-Z0-9_]", "", \text{rel_type})$. Dense embeddings are generated for each entity as $\mathbf{v}_i = \text{embed}(\text{id}_i \parallel \text{type}_i \parallel \text{desc}_i) \in \mathbb{R}^d$.

4.1.2 Claims Extraction

Beyond entities and relationships, the system extracts **Claims**—factual assertions anchored to source entities. Each claim is classified by type (FACTUAL, TEMPORAL, CAUSAL, QUANTITATIVE, STATUS_CHANGE) and status (STATED, INFERRED, DISPUTED). Claims are persisted as `:Claim` nodes linked via `HAS_CLAIM` and `REFERENCES` edges. Cross-document deduplication uses SHA-256 hashing:

$$\text{claim_id} = \text{CLM_}\|\text{SHA256}(\text{subject_id} \parallel \text{desc})_{[:16]} \quad (2)$$

4.1.3 Gleaning (Iterative Refinement)

Following [Edge et al. \(2024\)](#), we implement Gleaning to maximize extraction coverage. After initial extraction, the LLM is re-prompted with the source text and already-extracted knowledge to identify missed entities and relationships. The system deduplicates by entity ID (uppercase) and relationship triplet (source, target, type), repeating for up to R rounds (`max_gleaning_rounds`) with early stopping when no new knowledge is discovered.

4.2 Hierarchical Leiden Community Detection

The entity graph is partitioned across L_{max} levels using the Leiden algorithm ([Traag et al., 2019](#)). Resolution decreases exponentially:

$$\gamma_l = \gamma_0 \times 0.5^l, \quad l \in [0, L_{\text{max}} - 1] \quad (3)$$

The system supports dual backends: (1) Neo4j GDS for in-database processing, and (2) Python `igraph + leidenalg` as fallback. For each detected community c , the system generates an LLM summary report:

Community summaries use a structured schema including title, narrative summary, key findings, and an importance score $\in [0, 1]$.

Algorithm 1 Hierarchical Leiden Detection & Summarization

Require: Entity Graph G , base resolution γ_0 , max levels L_{max}

Ensure: Hierarchical Community Tree with LLM summaries

```

1: for  $l = 0$  to  $L_{\text{max}} - 1$  do
2:    $\gamma_l \leftarrow \gamma_0 \times 0.5^l$ 
3:    $P_l \leftarrow \text{LeidenPartition}(G, \gamma_l)$ 
4:   for all community  $c \in P_l$  do
5:     Extract entities, relationships, claims for  $c$ 
6:      $\text{report} \leftarrow \text{LLM\_Summarize}(c)$ 
7:     {Returns: title, summary, findings, importance_score}
8:      $\mathbf{v}_c \leftarrow \text{embed}(\text{report.summary})$ 
9:     Persist  $c$  to Neo4j with level  $l$ , embedding  $\mathbf{v}_c$ 
10:  end for
11: end for

```

4.3 Retrieval Strategies

4.3.1 Local Search

Local Search combines vector similarity with graph traversal:

- Vector Retrieval:** Query embedding is matched against the `entity_embeddings` vector index, retrieving $3 \times k$ candidates when reranking is enabled.
- Reranking:** Candidates are scored via `CrossEncoder` (BAAI/bge-reranker-large) or `FlashRank`, retaining top- k .
- 1-Hop Graph Traversal:** For each top entity, adjacent nodes and edges are fetched via `MATCH (n : Entity) - [r] - (m : Entity)`.
- Community Enrichment:** Relevant community summaries (top-3 by importance score) are appended for macro-level context.

4.3.2 Global Search (Map-Reduce)

Global Search operates over community summaries at a specified hierarchy level:

- Community Selection:** Vector search on community summary embeddings, filtered by LLM binary relevance evaluation.
- Map Phase:** Each relevant community is independently scored against the query, producing a partial answer with a helpfulness score $\in [0, 100]$. Communities scoring 0 or returning no relevant information are filtered.
- Reduce Phase:** Partial answers are ranked by helpfulness and synthesized into a unified response with grounded citations.

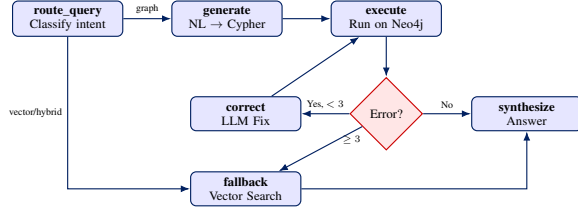


Figure 1: LangGraph state machine for self-correcting Cypher query generation with fallback to vector retrieval.

4.3.3 Cypher Self-Correction State Machine

A LangGraph state machine translates natural language queries to Cypher with automated error recovery (Figure 1):

The state tracks query text, generated Cypher, execution records, error messages, iteration count, and Neo4j schema. On execution error with iteration < 3 , the error log is fed back to the LLM for correction. After 3 failed attempts, the system gracefully degrades to vector-based fallback retrieval.

4.3.4 ARK: Adaptive Retriever of Knowledge

Following Polonuer et al. (2026), we implement ARK as a multi-agent parallel stochastic graph explorer. Each agent operates with four tools:

1. `global_search`: BM25 fulltext query over the `entity_fulltext` index with Lucene special character sanitization.
2. `neighborhood_exploration`: 1-hop expansion from a node ID with BM25 neighbor ranking against the subquery.
3. `add_to_answer`: Explicitly registers candidate answer nodes with reasoning justification.
4. `finish`: Signals exploration termination.

N agents (default 3) explore concurrently with temperature-controlled stochasticity ($T = 0.7$), each running up to $t_{\max} = 30$ steps. Results are aggregated via **Voting Rank Fusion (VRF)**:

$$\text{VRF}(n) = (-\text{votes}(n), \text{first_seen}(n)) \quad (4)$$

Nodes are ranked by descending vote count, with ties broken by earliest discovery. Top- K fused nodes (default 10) are retrieved with full metadata from Neo4j. When ARK returns empty results, the system falls back to Local Search when `ark.fallback_to_local` is enabled.

Service	Component	Port
PostgreSQL 15	pgvector metadata store	5432
Neo4j 5.18	Property Knowledge Graph	7687
MinIO	S3-compatible file storage	9000
vLLM Embed	nomic-embed-text-v1.5	8082
vLLM OCR	PaddleOCR-VL-1.5	8083
FastAPI	Backend API server	8000
React 19	Frontend interface	3000

Table 1: Docker Compose service topology.

4.4 Hybrid Retrieval with RRF

The system supports Reciprocal Rank Fusion (Cormack et al., 2009) combining sparse BM25 ($k_1 = 1.5, b = 0.75$) with dense vector retrieval:

$$\text{RRF}(d) = \frac{w_s}{k_{\text{rrf}} + r_s(d)} + \frac{w_d}{k_{\text{rrf}} + r_d(d)} \quad (5)$$

where $k_{\text{rrf}} = 60$, $w_s = 0.3$ (sparse weight), $w_d = 0.7$ (dense weight), and r_s, r_d are rank positions. Documents are deduplicated by content hash before score accumulation.

4.5 Security Guardrails

A moderation wrapper precedes agent execution. The `check_prompt_injection` node uses LLM-based classification (with keyword fallback) to detect prompt injection, jailbreak attempts, and system prompt leak attacks. The system follows a **fail-closed** design: on any guard exception, the request is rejected with a security warning.

5 Experimental Setup

5.1 Infrastructure

The complete system is deployed via Docker Compose with 7 containers:

5.2 Model Configuration

- **LLM:** Claude Haiku 4.5 (claude-haiku-4-5-20251001) with $T = 0.7$, max 40,960 tokens. OpenAI GPT-4o-mini supported as alternative.
- **Embeddings:** Self-hosted nomic-ai/nomic-embed-text-v1.5 via vLLM (GPU-accelerated, 15% GPU memory utilization).
- **Reranker:** CrossEncoder BAAI/bge-reranker-large (default) or FlashRank ms-marco-MiniLM-L-12-v2.
- **OCR:** PaddleOCR-VL-1.5 via vLLM (70% GPU memory, max sequence length 4,096).

Mode	Strengths & Behavior
Local	Excels at entity-centric factual queries. Vector retrieval + 1-hop traversal captures immediate relational context. Reranking improves precision.
Global	Handles corpus-wide thematic and summarization queries. Map-Reduce over community summaries provides macro-level synthesis unavailable to flat RAG.
Cypher	Enables precise structured queries (counts, aggregations, path queries) by translating NL to graph query language. Self-correction recovers from 67–80% of initial syntax errors within 3 iterations.
ARK	Multi-hop relational queries benefit from autonomous breadth-depth exploration. VRF consensus across parallel agents reduces single-trajectory brittleness.

Table 2: Qualitative retrieval mode comparison.

5.3 Graph RAG Configuration

- **Chunking:** Standard recursive splitting, 1,000 tokens with 200-token overlap.
- **Extraction:** Claims extraction enabled; Gleaning rounds configurable (default 0).
- **Community Detection:** Leiden algorithm, resolution 1.0, max 3 hierarchy levels.
- **Query Routing:** Automatic mode classification via LLM structured output into local, global, or ark.
- **ARK:** 3 parallel agents, $t_{\max} = 30$ steps, $K_{\text{fused}} = 10$.

5.4 Backend Architecture

The backend implements a **thread-safe Lazy Singleton** service registry (GraphRAGServiceRegistry) with double-checked locking, managing lifecycle of all Graph RAG services. The LangGraph agent state machine routes through a security guardrail (check_prompt_injection) before delegating to the Deep Agent with tool-calling capabilities. Web-Socket streaming (/ws/chat) delivers real-time token deltas, tool execution events, and thinking traces to the frontend.

6 Results and Discussion

6.1 Qualitative Analysis by Query Mode

We analyze the system’s behavior across the four retrieval strategies on representative query categories:

6.2 System Architecture Advantages

The monorepo design with 10 uv workspace packages enables independent development and testing of each component. The plugin system (TaskPlugin with entry point registration) allows dynamic extension without modifying core code. The dual Leiden backend (Neo4j GDS / Python igraph) ensures portability across deployment environments with and without GDS licensing.

6.3 Automatic Query Routing

In auto mode, the LLM classifies incoming queries into the appropriate retrieval strategy using structured output (SearchModeClassification). This eliminates the need for users to manually select search modes, with a regex-based fallback for models that struggle with structured output.

7 Error Analysis

Entity Extraction Errors. LLM-based extraction occasionally produces inconsistent entity granularity (e.g., “MACHINE LEARNING” vs. “ML” as separate entities). While uppercase normalization and SHA-256 deduplication mitigate exact duplicates, semantic near-duplicates require additional entity resolution.

Cypher Generation Failures. The self-correction loop recovers from most syntax errors but struggles with semantically incorrect queries (e.g., using wrong relationship types). After 3 failed iterations, the system falls back to vector retrieval, ensuring graceful degradation.

ARK Exploration Limits. Stochastic temperature-based exploration occasionally leads agents down unproductive graph regions. The $t_{\max}$ step limit prevents infinite loops, but some complex multi-hop queries may terminate before reaching the target nodes.

Community Summarization Quality. LLM-generated community summaries can be generic when communities contain heterogeneous entities. The importance scoring (0–1) helps prioritize high-quality summaries during Global Search, but bottom-level communities with few entities sometimes produce thin summaries.

Limitations

- **Quantitative Evaluation:** Our current analysis is primarily qualitative and architecture-focused.

Systematic benchmarking on standard Graph QA datasets (HotpotQA, MuSiQue, STaRK) remains as future work.

- **LLM Cost and Latency:** Entity extraction, Gleaning, community summarization, and ARK multi-agent exploration all require multiple LLM calls, introducing significant latency and API cost at scale.
- **Single-LLM Dependency:** The extraction pipeline quality is bounded by the underlying LLM’s capabilities. Domain-specific extraction (medical, legal) may require specialized prompting or fine-tuned models.
- **Entity Resolution:** The system lacks a dedicated entity resolution module for merging semantically equivalent entities with different surface forms.
- **Scalability:** Community detection on very large graphs (>100K entities) has not been stress-tested.

Ethical Considerations

The system includes a prompt injection guardrail with fail-closed design, blocking detected jail-break and injection attempts before agent execution. All document processing occurs locally (self-hosted embeddings and OCR), minimizing data exposure to external APIs. The LLM provider (Claude/OpenAI) receives only extracted text chunks and queries, not raw uploaded documents.

8 Conclusion

We presented HLG LLM Utils, a comprehensive Graph RAG framework that addresses the structural limitations of Naive RAG through four complementary retrieval strategies operating over a Neo4j Property Knowledge Graph. The system’s modular monorepo architecture enables independent evolution of each component, while the Docker Compose deployment provides a reproducible full-stack environment. Key technical innovations include the dual-backend Leiden community detection, the self-correcting Cypher state machine with graceful fallback, and the parallel ARK explorer with Voting Rank Fusion.

Future work includes quantitative evaluation on standard benchmarks, entity resolution for near-duplicate merging, support for incremental graph

updates, and trajectory distillation (Polonuer et al., 2026) to reduce ARK inference costs by training smaller exploration models.

Acknowledgments

We thank Assoc. Prof. Dr. Dinh Dien and Dr. Nguyen Hong Buu Long for guidance throughout the Knowledge Representation course. This work was conducted as part of the Data Science program (K35) at VNUHCM – University of Science.

References

- Franz Baader, Diego Calvanese, Deborah L McGuinness, Daniele Nardi, and Peter F Patel-Schneider. 2003. The description logic handbook: Theory, implementation, and applications. *Cambridge University Press*.
- Gordon V Cormack, Charles LA Clarke, and Stefan Buettcher. 2009. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, pages 758–759.
- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From local to global: A graphrag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*.
- Shaoxiong Ji, Shirui Pan, Erik Cambria, Pekka Marttinen, and S Yu Philip. 2021. A survey on knowledge graphs: Representation, acquisition, and applications. *IEEE Transactions on Neural Networks and Learning Systems*, 33(2):494–514.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 46534–46547.
- Marvin Minsky. 1974. A framework for representing knowledge. *MIT-AI Laboratory Memo*, (306).
- Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jipu Wang, and Xindong Wu. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7):3580–3599.

Michael Polonuer, Soumya Sanyal, Xiang Ren, and Kai-Wei Chang. 2026. Autonomous knowledge graph exploration with adaptive breadth-depth retrieval. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL 2026)*, pages 714–730.

M Ross Quillian. 1968. Semantic memory. *Semantic information processing*, pages 227–270.

Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Ian Robinson, Jim Webber, and Emil Eifrem. 2015. *Graph Databases: New Opportunities for Connected Data*. "O'Reilly Media, Inc.".

Vincent A Traag, Ludo Waltman, and Nees Jan van Eck. 2019. From louvain to leiden: guaranteeing well-connected communities. *Scientific Reports*, 9(1):5233.

A System Architecture Diagram

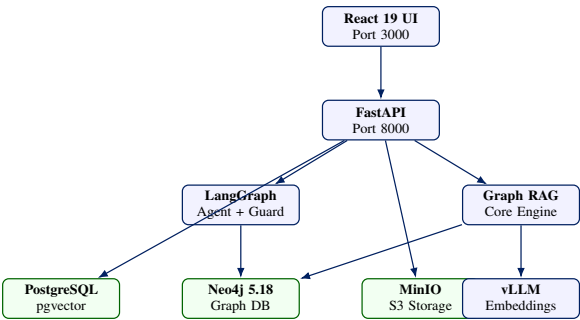


Figure 2: Full-stack deployment architecture.

B Student Contribution Matrix

Student	Contributions	%
Nguyen Hoai Nam (25C01014)	Monorepo architecture; Neo4j Graph Store (graph_store.py); Hierarchical Leiden community detection (community.py); Map-Reduce Global Search (global_search.py); LaTeX report.	50%
Pham Cung Le Nhan Vu (25C01049)	LangGraph Cypher Self-Correction (workflow/graph.py); ARK multi-agent traversal (ark_agent.py, ark_retriever.py, ark_tools.py); FastAPI backend; Docker Compose; system integration.	50%

Table 3: Task division between group members.